

Overparameterized Neural Forecasting with DSGE Priors

Yun Dou*

May 29, 2026

Abstract

This paper asks whether structural prior information can help an overparameterized neural network exploit nonlinear flexibility in small-sample macroeconomic forecasting. I pretrain a neural network on DSGE-simulated data and fine-tune it on real macroeconomic data. The DSGE-informed network is stable and competitive, but its forecast paths closely resemble those of low-lag BVAR and DSGE-VAR. This suggests that the network mainly learns a low-dimensional, approximately linear Bayesian-shrinkage pattern rather than additional nonlinear predictive signals. DSGE priors are useful relative to no prior and simpler statistical priors, but mainly as regularization and prior-induced initialization rather than as a source of new nonlinear predictive content.

1 Introduction

Macroeconomic forecasting faces a fundamental tension. On the one hand, researchers would like to use richer information sets and more flexible forecasting models in order to capture potentially complex dynamic relationships among macroeconomic variables. On the other hand, macroeconomic time series are typically short, so expanding the number of variables, lag lengths, or model complexity can quickly generate substantial estimation variance (Stock and Watson, 2002; Giannone et al., 2015; Sims, 1980). Within the traditional framework in which the number of parameters p is smaller than the sample size n , the central contribution of BVAR (Doan et al., 1984) and DSGE-VAR (Del Negro and Schorfheide, 2004) methods is to improve the bias-variance tradeoff by introducing statistical or structural priors. These methods are effective in stabilizing VAR forecasts, but they remain essentially linear forecasting frameworks. This raises a natural question: does further progress in macroeconomic forecasting require moving beyond the conventional $p < n$ model space?

The double descent phenomenon in modern deep learning (Belkin et al., 2019; Nakkiran et al., 2021) suggests a different perspective. In some prediction problems, having more parameters than observations does not necessarily lead to poor generalization. With suitable inductive bias and regularization, an overparameterized model may still learn an effective forecasting function. Motivated by this idea, this paper asks whether prior information can help an overparameterized neural network use its functional flexibility in a stable way, learn nonlinear predictive signals that traditional linear Bayesian predictors fail to capture, and thereby outperform standard forecasting benchmarks in a small-sample macroeconomic environment.

To answer this question, this paper studies a DSGE-informed neural forecasting framework. The neural network is first pretrained on DSGE-simulated data and then fine-tuned on real macroeconomic data. DSGE pretraining can be interpreted as an implicit prior. The simulated DSGE data are not treated as substitutes for real data; rather, pretraining initializes the neural network parameters, or equivalently its forecasting function, in an economically meaningful region. Fine-tuning then allows the real data to revise this prior-induced initialization.

*University of Chicago, Master of Arts in Economics. I am deeply grateful to Professor Mikkel Plagborg-Møller for his invaluable guidance and support. All remaining errors are my own.

The neural network uses a bottleneck multi-head architecture. The economic interpretation is that, although researchers can observe many macroeconomic variables and their lags, the underlying macro states relevant for forecasting may be low-dimensional (Stock and Watson, 2002). The wide hidden layers at the front of the network extract candidate features from lagged macroeconomic variables. The bottleneck layer compresses this information into a small number of latent macro states. The variable-specific output heads then map these latent states into forecasts for different macroeconomic variables. This design is inspired by the latent-state structure of DSGE models and seeks to embed an economically interpretable low-dimensional representation into a flexible neural network.

The first main result is that, in an overparameterized $p > n$ setting, a DSGE prior can make the neural network stable and competitive with traditional Bayesian benchmarks. Using a small-scale three-variable New Keynesian DSGE model as the prior source, this paper compares the DSGE-informed neural network with BVAR and DSGE-VAR for forecasting GDP growth, inflation, and the short-term interest rate. The results show that the DSGE prior helps stabilize the neural network and brings its forecasting performance close to that of BVAR and DSGE-VAR. In particular, when lag length increases, the forecast errors of traditional VAR-type models tend to rise more strongly, while the DSGE-informed neural network remains relatively stable. However, the DSGE-informed neural network only matches the traditional benchmarks; it does not clearly outperform them. Relative to low-lag BVAR and DSGE-VAR, the forecast errors of DSGE-NN-MH are usually close, but not systematically lower.

Forecast paths further explain this finding. The forecasts produced by the DSGE-informed neural network are very similar to those produced by BVAR and DSGE-VAR. In other words, the forecasting pattern learned by the neural network is close to the signal captured by these linear Bayesian predictors.

This conclusion is not primarily driven by random variation in neural network training, nor does it depend on one particular DSGE prior or neural-network architecture. Forecasts across different random seeds are similar, suggesting that the main result is not an artifact of initialization noise or optimization noise. Moreover, when this paper uses a richer DSGE prior, expands the information set, or replaces the bottleneck multi-head architecture with a standard fully connected neural network, the core result remains similar: the neural network continues to behave like low-lag BVAR and DSGE-VAR, without producing clear forecast dominance.

The interpretation is that, in the low-frequency and low-dimensional macroeconomic environment considered here, the signal-to-noise ratio may be low and the amount of useful predictive information may be limited. Equivalently, the variables used in this paper may already be highly compressed macroeconomic signals, so the stable and useful forecasting structure may itself be low-dimensional and approximately linear. In such an environment, linear Bayesian predictors such as BVAR and DSGE-VAR may already capture most of the relevant predictive signal. Even though a neural network has greater functional flexibility, it may not be able to learn additional nonlinear predictive information from the limited sample.

This paper also studies the role of the DSGE prior itself. The results show that the DSGE prior is useful relative to a no-prior raw neural network, a historical-mean prior, and a Minnesota-style random-walk prior (Litterman, 1986). The historical-mean prior tends to pull highly persistent macroeconomic variables toward their long-run means, which can generate substantial misspecification. The random-walk prior captures the idea of a no-change forecast, but it lacks cross-variable dynamics and shock propagation. The raw neural network is more unstable when fine-tuned on a small real-data sample.

At the same time, the role of the DSGE prior should be interpreted cautiously. This paper does not find strong evidence of a conditional bias–variance tradeoff. A natural conjecture is that if the prior is too tight, the model may inherit DSGE misspecification bias; if the prior is too loose, the model may collapse into a high-variance raw neural network; and if the prior tightness is intermediate, the model may combine structural discipline with data-driven flexibility and

outperform BVAR or DSGE-VAR. The empirical results do not support this stronger mechanism. Instead, the DSGE prior mainly provides a useful starting point. It helps the flexible model learn low-dimensional, approximately linear forecasting signals that are very similar to those captured by BVAR and DSGE-VAR. Put differently, the DSGE prior helps the model reach a forecasting region that existing benchmarks can also reach, rather than guiding it toward a clearly superior new predictor.

A further limitation is that DSGE-NN does not naturally provide credible forecast uncertainty or density forecasts. Although the DSGE-informed neural network can produce competitive point forecasts, it is less straightforward to obtain convincing measures of predictive uncertainty than in BVAR or DSGE-VAR (Gal and Ghahramani, 2016). Since uncertainty is central to macroeconomic forecasting, a neural forecasting model that delivers only competitive but not clearly superior point forecasts, while lacking reliable uncertainty quantification, has limited practical advantage.

This study also points to two directions for future research. First, to fully exploit the potential of overparameterized neural networks in macroeconomic forecasting, future work may need to use higher-dimensional, higher-frequency, and more granular information sets rather than only a small number of low-frequency macroeconomic variables. Similar to central-bank nowcasting and forecasting practice (Bok et al., 2018), future models could incorporate richer financial, industrial, labor-market, price, consumption, and high-frequency activity data, and then use DFM or neural representation learning to compress these data into useful macro states. DSGE-informed neural networks remain promising in this direction because this paper shows that they can be stabilized in a $p > n$ environment. Second, future extensions of DSGE-NN should place greater emphasis on forecast uncertainty and density forecasting. Rather than improving point forecasts alone, a more promising direction may be to develop Bayesian transfer learning or probabilistic neural forecasting frameworks, so that structural priors help characterize not only predictive means but also predictive uncertainty.

2 DSGE Prior: Estimation and Simulation

This section describes how I construct the structural prior used in the neural forecasting exercise. The prior is based on a small-scale New Keynesian DSGE model with three observables: output growth, inflation, and the short-term nominal interest rate. The model specification and the Bayesian state-space estimation procedure follow the standard treatment in Herbst and Schorfheide (2015). The purpose of the DSGE model in this paper is not to provide a fully specified data-generating process for the U.S. economy. Rather, it provides a disciplined source-domain model that summarizes basic monetary transmission, inflation dynamics, and interest-rate smoothing. I estimate this model using Bayesian methods, generate artificial data from the estimated structure, and use these simulated paths as pretraining data for the neural network in the next section.

2.1 A Small-Scale New Keynesian DSGE Model

The DSGE prior is based on a standard three-equation New Keynesian model. Let $\hat{y}_t$ denote the output gap, $\hat{\pi}_t$ inflation, and $\hat{R}_t$ the nominal interest rate, all expressed as log deviations from steady state. The log-linearized equilibrium conditions are

$$\hat{y}_t = E_t \hat{y}_{t+1} - \frac{1}{\tau} \left(\hat{R}_t - E_t \hat{\pi}_{t+1} - E_t \hat{z}_{t+1} \right) + \hat{g}_t - E_t \hat{g}_{t+1}, \quad (1)$$

$$\hat{\pi}_t = \beta E_t \hat{\pi}_{t+1} + \kappa(\hat{y}_t - \hat{g}_t), \quad (2)$$

$$\hat{R}_t = \rho_R \hat{R}_{t-1} + (1 - \rho_R) [\psi_1 \hat{\pi}_t + \psi_2 (\hat{y}_t - \hat{g}_t)] + \varepsilon_t^R. \quad (3)$$

The first equation is the intertemporal IS equation, the second is the New Keynesian Phillips curve, and the third is a Taylor-type monetary policy rule with interest-rate smoothing. The exogenous processes are

$$\hat{g}_t = \rho_g \hat{g}_{t-1} + \varepsilon_t^g, \quad (4)$$

$$\hat{z}_t = \rho_z \hat{z}_{t-1} + \varepsilon_t^z, \quad (5)$$

where $\varepsilon_t = (\varepsilon_t^R, \varepsilon_t^g, \varepsilon_t^z)'$ is Gaussian with diagonal covariance matrix

$$Q(\theta) = \text{diag}(\sigma_R^2, \sigma_g^2, \sigma_z^2).$$

The structural parameter vector is

$$\theta = (\tau, \kappa, \psi_1, \psi_2, \rho_R, \rho_g, \rho_z, r_A, \pi_A, \gamma_Q, \sigma_R, \sigma_g, \sigma_z)'$$

The discount factor is not estimated directly. It is implied by the annualized steady-state real rate,

$$\beta = \frac{1}{1 + r_A/400}.$$

For a given value of θ , I write the model in Sims (2002)' canonical linear rational expectations form,

$$\Gamma_0(\theta)s_t = \Gamma_1(\theta)s_{t-1} + \Psi(\theta)\varepsilon_t + \Pi(\theta)\eta_t, \quad (6)$$

where η_t collects one-step-ahead expectational errors. The state vector is

$$s_t = \left(\hat{y}_t, \hat{\pi}_t, \hat{R}_t, \varepsilon_t^R, \hat{g}_t, \hat{z}_t, E_t \hat{y}_{t+1}, E_t \hat{\pi}_{t+1} \right)'$$

I solve (6) using a QZ-based Sims solver (Sims, 2002). Parameter values that fail the determinacy and stability conditions are assigned zero posterior density. When a unique stable solution exists, the model implies

$$s_t = G_1(\theta)s_{t-1} + G_\varepsilon(\theta)\varepsilon_t. \quad (7)$$

Because observed output growth depends on $\hat{y}_t - \hat{y}_{t-1}$, I augment the state vector with lagged output,

$$a_t = (s'_t, \hat{y}_{t-1})',$$

and obtain the state transition equation

$$a_t = T(\theta)a_{t-1} + R(\theta)\varepsilon_t. \quad (8)$$

The observed data vector is

$$Y_t = (YGR_t, INFL_t, INT_t)',$$

where YGR_t is output growth, $INFL_t$ is annualized inflation, and INT_t is the annualized nominal interest rate. The measurement equations are

$$YGR_t = \gamma_Q + 100(\hat{y}_t - \hat{y}_{t-1} + \hat{z}_t), \quad (9)$$

$$INFL_t = \pi_A + 400\hat{\pi}_t, \quad (10)$$

$$INT_t = \pi_A + r_A + 4\gamma_Q + 400\hat{R}_t. \quad (11)$$

Equations (8)–(11) define a linear Gaussian state-space system. This representation is the bridge between the DSGE model and the likelihood used in Bayesian estimation.

Table 1: Posterior summary of the small-scale NK DSGE parameters

Parameter	Mean	SD	5%	95%
τ	2.7750	0.5341	1.9637	3.7176
κ	0.5084	0.1385	0.3160	0.7615
ψ_1	1.1311	0.0847	1.0152	1.2873
ψ_2	0.0902	0.0731	0.0110	0.2336
ρ_R	0.6308	0.0505	0.5434	0.7099
ρ_g	0.9803	0.0101	0.9615	0.9937
ρ_z	0.8620	0.0212	0.8262	0.8957
r_A	0.3034	0.1452	0.1069	0.5763
π_A	2.6845	0.4914	1.8619	3.4801
γ_Q	0.2557	0.0780	0.1290	0.3834
σ_R	0.002666	0.000226	0.002324	0.003065
σ_g	0.013710	0.001070	0.012070	0.015570
σ_z	0.002124	0.000258	0.001742	0.002587

2.2 Bayesian Estimation

The DSGE parameters are estimated using the observed three-variable macroeconomic data prior to 1990Q1. Given the state-space representation above, the likelihood is evaluated with the Kalman filter. Let $v_t(\theta)$ be the one-step-ahead forecast error and $F_t(\theta)$ its conditional covariance matrix. The log likelihood is

$$\log p(Y | \theta) = -\frac{1}{2} \sum_{t=1}^T [n \log(2\pi) + \log |F_t(\theta)| + v_t(\theta)' F_t(\theta)^{-1} v_t(\theta)], \quad (12)$$

where $n = 3$. The posterior distribution is

$$p(\theta | Y) \propto p(Y | \theta) p(\theta). \quad (13)$$

The prior $p(\theta)$ is weakly informative and imposes the standard economic restrictions: positive preference, policy, and shock-scale parameters; persistence parameters inside the unit interval; and economically plausible steady-state values. The full prior specification is reported in Appendix A.

I sample from the posterior using random-walk Metropolis–Hastings in an unconstrained parameter space. Positive parameters are log-transformed, persistence parameters are logit-transformed, and the steady-state growth parameter is left unrestricted. If x denotes the unconstrained parameter vector and $\theta = T(x)$ the mapping back to structural parameters, the target density in transformed space is

$$\log p(x | Y) = \log p(Y | \theta(x)) + \log p(\theta(x)) + \log \left| \frac{\partial \theta}{\partial x} \right|. \quad (14)$$

The Jacobian term is necessary because the prior is specified on the structural parameter vector θ , while the Markov chain is simulated in the transformed space x .

The posterior simulation uses four chains, each with 100,000 draws. After discarding 5,000 burn-in draws per chain and thinning by a factor of four, the final posterior sample contains 23,750 retained draws per chain. The acceptance rates are close to 23 percent across all chains, and the estimated $\hat{R}$ statistics are approximately one for all parameters. Table 1 reports the posterior summary used in the subsequent simulation step.

In the forecasting exercise below, I use the posterior mean vector,

$$\bar{\theta} = E[\theta | Y],$$

as the center of the DSGE simulation step. Thus, the DSGE model enters the paper through a Bayesian estimation procedure, but the subsequent simulation does not use the full posterior predictive distribution.

2.3 Simulation of DSGE-Implied Training Data

The neural network is pretrained on artificial data generated from the estimated DSGE model. The simulation is centered at the posterior mean $\bar{\theta}$, with small perturbations around this value to generate heterogeneous but structurally related paths. Specifically, for each simulated path i , I construct a parameter vector θ_i by perturbing $\bar{\theta}$ in the admissible transformed parameter space and retaining only parameter values that produce a stable unique solution. I then solve the DSGE model, simulate the state transition equation (8), and map the simulated states into observables using (9)–(11).

The baseline simulation uses 1,000 independent paths. Each path is simulated for 200 quarters, and the first 100 quarters are discarded as burn-in. The remaining observations are mapped into the variables used in the forecasting exercise: YGR_t is matched to GDP growth, $INFL_t/4$ is matched to quarterly GDP deflator inflation, and $INT_t/4$ is matched to the quarterly short-term interest rate. Finally, the simulated series are affinely rescaled using the mean and standard deviation of the corresponding real data before 1980. This scaling step aligns the simulated DSGE data with the units of the real forecasting data without changing the temporal dependence implied by the structural model.

It is important to clarify what this simulation step does and does not do. I do not generate one simulated data set for each MCMC posterior draw, and therefore I do not use the full posterior predictive distribution of the DSGE model. Instead, MCMC is used to estimate a plausible posterior center for the structural model, and the simulated paths are generated around this center with small parameter perturbations. This choice keeps the pretraining data structurally disciplined while avoiding excessive simulation noise from unstable or weakly identified parts of the posterior. The resulting simulated data set is denoted by

$$\mathcal{D}^{sim} = \{(X_{t,L}^{sim}, Y_{t+1}^{sim})\},$$

and is used as the source-domain sample for the DSGE-informed neural network described in Section 3.

3 DSGE-NN-MH Forecasting Model

This section describes how the DSGE-implied simulated data are used to discipline an overparameterized neural forecasting model. The model is denoted by DSGE-NN-MH, where “NN” refers to the neural network forecasting function and “MH” refers to the multi-head output structure. The central idea is simple: DSGE simulations provide source-domain data for pretraining, and real macroeconomic data then fine-tune the pretrained forecasting function at each forecast origin. In this sense, the DSGE model enters the neural network not as a literal data-generating process for the U.S. economy, but as a prior-inducing source model.

Let $Y_t \in R^n$ denote the vector of target variables. In the baseline three-variable exercise,

$$Y_t = (GDP_t, \pi_t, i_t)',$$

where GDP_t is GDP growth, π_t is GDP deflator inflation, and i_t is the short-term interest rate. For a lag length L , define the input vector as

$$X_{t,L} = (Y'_{t-1}, Y'_{t-2}, \dots, Y'_{t-L})'. \quad (15)$$

The forecasting task is one-step-ahead direct prediction,

$$Y_{t+1} = f(X_{t,L}) + e_{t+1}. \quad (16)$$

3.1 Architecture

The DSGE-NN-MH forecasting function is a feedforward neural network with shared hidden layers and variable-specific output heads. Let ϕ collect the parameters in the shared part of the network, including the encoder, bottleneck layer, and shared decoder. Let

$$\omega = (\omega_1, \dots, \omega_n)$$

collect the parameters in the n output heads.

The shared part of the network maps lagged macroeconomic variables into a low-dimensional latent representation:

$$h_t = E_\phi(X_{t,L}), \quad h_t \in R^k, \quad (17)$$

where $k = 4$ in the baseline specification. The bottleneck h_t is the main architectural restriction. It reflects the view that, although many lagged variables can be observed, the information relevant for forecasting a small set of macroeconomic aggregates may be summarized by a low-dimensional set of latent macro states.

Given this shared representation, each target variable is predicted by a separate output head:

$$\hat{Y}_{j,t+1} = m_{j,\omega_j}(h_t), \quad j = 1, \dots, n. \quad (18)$$

Thus the full forecasting function is

$$f_{\phi,\omega}(X_{t,L}) = (m_{1,\omega_1}(E_\phi(X_{t,L})), \dots, m_{n,\omega_n}(E_\phi(X_{t,L})))'. \quad (19)$$

This architecture can be interpreted as a weak neural analogue of a low-dimensional state-space or dynamic-factor representation. The shared layers extract a common macroeconomic representation, the bottleneck restricts its effective dimension, and the output heads allow GDP growth, inflation, and the interest rate to have variable-specific forecasting mappings. The structure does not impose the DSGE law of motion exactly. Instead, the DSGE model disciplines the network through pretraining, while the bottleneck provides an additional representation prior.

3.2 Pretraining on DSGE Simulations

Let

$$\mathcal{D}_L^{sim} = \{(X_{t,L}^{sim}, Y_{t+1}^{sim})\}$$

denote the source-domain sample generated from the estimated DSGE model. The pretraining step chooses the network parameters by minimizing the simulated-data prediction loss:

$$(\hat{\phi}^{pre}, \hat{\omega}^{pre}) = \arg \min_{\phi, \omega} \frac{1}{|\mathcal{D}_L^{sim}|} \sum_{(X_{t,L}^{sim}, Y_{t+1}^{sim}) \in \mathcal{D}_L^{sim}} \|Y_{t+1}^{sim} - f_{\phi,\omega}(X_{t,L}^{sim})\|_2^2. \quad (20)$$

This step is where the DSGE prior enters the neural network. The simulated observations are not treated as additional real data. Rather, they move the randomly initialized network toward a forecasting function that reflects the dynamic restrictions of the estimated New Keynesian model. In Bayesian language, pretraining places the neural network in a prior-induced region of function space before it is exposed to the small real-data forecasting sample.

3.3 Rolling Fine-Tuning on Real Data

After pretraining, the network is fine-tuned on real macroeconomic data. For each forecast origin T , only the previous $W = 80$ quarters are used as the real-data training window. Starting from the pretrained parameters, the fine-tuned parameters solve

$$(\hat{\phi}_T, \hat{\omega}_T) = \arg \min_{\phi, \omega} \frac{1}{W} \sum_{t=T-W}^{T-1} \|Y_{t+1} - f_{\phi,\omega}(X_{t,L})\|_2^2, \quad (\phi, \omega) \text{ initialized at } (\hat{\phi}^{pre}, \hat{\omega}^{pre}). \quad (21)$$

The one-step-ahead forecast is then

$$\hat{Y}_{T+1|T} = f_{\hat{\phi}_T, \hat{\omega}_T}(X_{T,L}). \quad (22)$$

This rolling-origin design keeps the evaluation genuinely out of sample: no information after the forecast origin is used when producing the forecast. It also clarifies the role of the prior. The pretrained network provides an initial forecasting rule, and the rolling real-data window determines how much that rule is revised before each forecast. The effective tightness of the implicit prior is governed by fine-tuning choices such as the learning rate, number of epochs, batch size, and whether parts of the network are frozen. In the baseline specification, the encoder is not frozen, so the real data can adjust both the latent representation and the output heads. The prior remains influential because the fine-tuning sample is small and optimization starts from the DSGE-pretrained parameters.

3.4 Forecasting Inputs and Empirical Variants

The baseline experiment uses the same three variables as both inputs and outputs: lags of GDP growth, GDP deflator inflation, and the short-term interest rate are used to forecast the same three variables. The empirical analysis evaluates lag lengths $L = 1, \dots, 8$. This lag grid is not intended to make lag length a separate theoretical object. Rather, it asks whether DSGE pretraining and the bottleneck representation make the neural forecast less vulnerable to the usual small-sample variance cost of adding lagged predictors.

I also consider an extended-input version in which the output remains the same three target variables, but the input contains lags of eight macroeconomic variables. This exercise is used later as a robustness and interpretation check. It asks whether adding a richer information set and a richer DSGE-related source domain creates additional forecastable signal for the target variables. As shown in the results, the answer is not systematically positive in the current implementation.

Thus, the main comparison is not between two different neural-network theories, but between different sources of forecasting information. The baseline version asks whether DSGE pretraining can stabilize an overparameterized neural network in a small three-variable system. The extended-input version asks whether the same framework benefits from a broader macroeconomic information set. In both cases, the core object remains the same: a DSGE-pretrained neural forecasting function that is fine-tuned on a small rolling real-data sample.

4 Benchmark Models

This section defines the benchmark models used to evaluate DSGE-NN-MH. The purpose is not only to compare forecast errors, but also to clarify what kind of prior information the neural network is being compared against. The main linear benchmarks are a BVAR with a Minnesota-style statistical prior and a DSGE-VAR with a structural prior constructed from the same DSGE simulations used for neural-network pretraining. I also use a raw neural network to isolate the role of DSGE pretraining.

All models are evaluated under the same rolling pseudo-out-of-sample design described in Section 5. At each forecast origin, the model is estimated or fine-tuned using only information available at that date and then used to produce a one-quarter-ahead forecast.

Let $Y_t \in R^n$ denote the target vector and let $X_{t,L}$ denote the lagged input vector. For the linear benchmarks, define

$$x_{t,L} = (1, X'_{t,L})',$$

and consider the linear forecasting model

$$Y_{t+1} = B'x_{t,L} + u_{t+1}. \quad (23)$$

The benchmarks differ in how they discipline the estimation of B in the small rolling sample.

4.1 Bayesian VAR with Minnesota Prior

The BVAR benchmark uses a Minnesota-style prior. The prior shrinks the VAR toward a parsimonious persistence-based forecasting rule: own first lags receive the main prior weight, while cross-variable coefficients and longer-lag coefficients are shrunk toward zero. This is the standard statistical-prior solution to the small-sample problem in macroeconomic VAR forecasting.

Conditional on the innovation covariance matrix, the coefficient prior is Gaussian:

$$\text{vec}(B) \mid \Sigma_u \sim N(\text{vec}(B_0), \Sigma_u \otimes \Omega_B(\lambda_B)), \quad (24)$$

where B_0 is the Minnesota prior mean and $\Omega_B(\lambda_B)$ is the prior covariance matrix. The hyperparameter λ_B controls prior looseness: smaller values imply tighter shrinkage toward B_0 , while larger values move the posterior closer to the unrestricted rolling-sample estimate. The posterior mean can be written as

$$\hat{B}_{\text{BVAR}}(\lambda_B) = (X'X + \Omega_B(\lambda_B)^{-1})^{-1} (X'Y + \Omega_B(\lambda_B)^{-1}B_0), \quad (25)$$

up to the usual covariance scaling. The tightness parameter is selected on the validation period and then fixed during the test-period evaluation.

This benchmark is important because it shows what can be achieved by a well-regularized linear Bayesian predictor without structural DSGE information.

4.2 DSGE-VAR

The DSGE-VAR benchmark uses the DSGE model as a structural prior for a linear VAR. Conceptually, it asks how much forecast performance can be obtained by incorporating the DSGE-implied dynamics into a linear forecasting model, rather than into a neural network.

I implement DSGE-VAR using the simulated DSGE data constructed in Section 2. For each lag length L , I construct simulated regressors and targets,

$$\mathcal{D}_L^{\text{sim}} = \{(X_{t,L}^{\text{sim}}, Y_{t+1}^{\text{sim}})\},$$

in the same format as the real forecasting sample. Let

$$S_{xx} = \frac{1}{T} X'X, \quad S_{xy} = \frac{1}{T} X'Y$$

denote the real-sample moment matrices in a rolling window, and let

$$S_{xx}^{\text{sim}} = \frac{1}{T^{\text{sim}}} X^{\text{sim}'} X^{\text{sim}}, \quad S_{xy}^{\text{sim}} = \frac{1}{T^{\text{sim}}} X^{\text{sim}'} Y^{\text{sim}}$$

denote the corresponding simulated DSGE moments. The DSGE-VAR coefficient estimate is

$$\hat{B}_{\text{DSGE-VAR}}(\lambda_D) = (S_{xx} + \lambda_D S_{xx}^{\text{sim}})^{-1} (S_{xy} + \lambda_D S_{xy}^{\text{sim}}). \quad (26)$$

The scalar λ_D controls the weight placed on the DSGE prior. When $\lambda_D = 0$, the estimator uses only the real rolling sample. As λ_D increases, the linear forecast is pulled more strongly toward the projection implied by the DSGE-simulated data. Thus DSGE-VAR and DSGE-NN-MH use the same source of structural information but incorporate it differently: DSGE-VAR shrinks linear VAR coefficients toward the DSGE-implied linear projection, while DSGE-NN-MH initializes a nonlinear forecasting function in a DSGE-implied region of function space.

This implementation is close in spirit to the DSGE-VAR literature, but it uses simulated sample moments rather than analytical DSGE population moments. This choice keeps the comparison with DSGE-NN-MH transparent, because both models use the same DSGE simulation as the source-domain prior information.

4.3 Raw Neural Network and Prior Variants

The raw neural network uses the same architecture as DSGE-NN-MH but does not use DSGE pretraining. At each forecast origin, it starts from random weights and is trained only on the rolling real-data window:

$$(\hat{\phi}_T^{raw}, \hat{\omega}_T^{raw}) = \arg \min_{\phi, \omega} \frac{1}{W} \sum_{t=T-W}^{T-1} \|Y_{t+1} - f_{\phi, \omega}(X_{t,L})\|_2^2. \quad (27)$$

This benchmark isolates the value of the DSGE-induced initialization from the value of the neural-network architecture itself.

For mechanism checks, I also consider neural networks pretrained on simpler prior-generating processes, including a historical-mean prior and a random-walk prior. These variants are not the main forecasting benchmarks. They are used to interpret whether the DSGE prior is useful because it provides economically meaningful cross-variable dynamics and shock propagation, rather than merely because any pretraining stabilizes the network.

5 Empirical Evidence

This section presents the empirical forecasting evidence for DSGE-NN-MH and organizes the results according to the main argument in the Introduction. I first compare DSGE-NN-MH with BVAR and DSGE-VAR in the baseline forecasting horse race. I then use forecast paths to examine whether the neural network learns a forecasting pattern materially different from linear Bayesian predictors. I next check whether the main conclusion depends on random seeds, the information set, or the neural-network architecture. Finally, I study the role of the DSGE prior itself.

5.1 Empirical Setup

The baseline target vector is

$$Y_t = (GDP_t, \pi_t, i_t)',$$

where GDP_t denotes GDP growth, π_t denotes GDP deflator inflation, and i_t denotes the short-term interest rate. The baseline specification uses lags of these same three variables as inputs and produces one-quarter-ahead forecasts. For lag length L , the input vector is

$$X_{t,L} = (Y'_{t-1}, Y'_{t-2}, \dots, Y'_{t-L})',$$

and each model produces a forecast $\hat{Y}_{T+1|T}$ at forecast origin T .

All models are evaluated using the same rolling pseudo-out-of-sample design. The validation period is 1980Q1–1995Q4 and is used to choose the prior tightness parameters for BVAR and DSGE-VAR and to fix the main neural-network training settings. The test period starts in 1996Q1 and is used only for final out-of-sample evaluation. At each forecast origin, each model is estimated or fine-tuned using the previous $W = 80$ quarters of data. The lag grid is

$$L = 1, \dots, 8.$$

Forecast performance is summarized by squared forecast errors. For model m , variable j , and forecast origin T , define

$$SE_{j,T}^{(m)} = \left(Y_{j,T+1} - \hat{Y}_{j,T+1|T}^{(m)} \right)^2.$$

The main text reports median and upper-quartile squared errors. Lower-quartile errors and other supplementary figures are reported in Appendix D.

5.2 Baseline Horse Race: Stability without Clear Dominance

The first empirical question is whether the DSGE prior can make an overparameterized neural network stable and competitive with BVAR and DSGE-VAR in a small-sample macroeconomic forecasting environment. Figures 1 and 2 compare DSGE-NN-MH, BVAR, and DSGE-VAR across lag lengths.

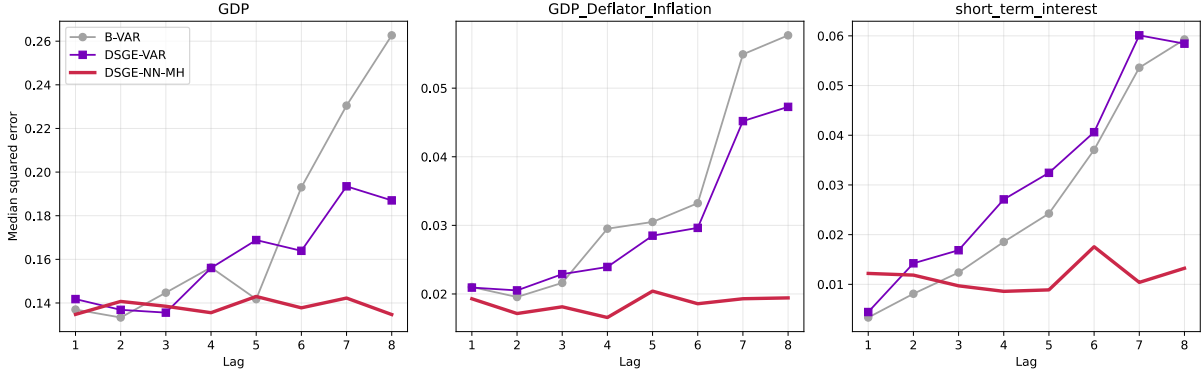


Figure 1: **Baseline horse race: median squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to GDP growth, GDP deflator inflation, and the short-term interest rate. Each line reports the median squared forecast error across test-period forecast origins for one model.

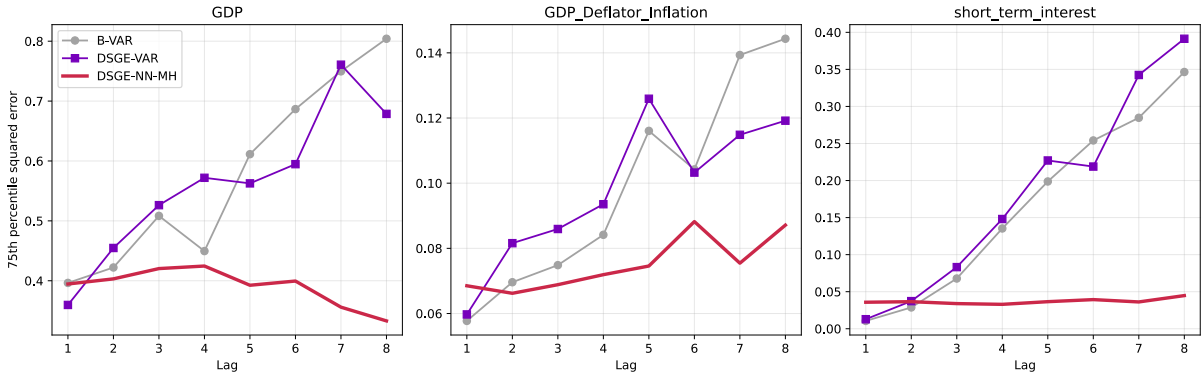


Figure 2: **Baseline horse race: 75th percentile squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Each line reports the 75th percentile squared forecast error across test-period forecast origins for one model.

The main advantage of DSGE-NN-MH is its relative stability across lag lengths. As L increases, BVAR and DSGE-VAR forecast errors tend to rise, especially for GDP growth and the short-term interest rate. In contrast, the DSGE-NN-MH error curves are relatively flat. This suggests that DSGE pretraining and the bottleneck representation reduce the effective variance cost of adding more lagged predictors.

However, this result should be interpreted cautiously. DSGE-NN-MH is competitive, but not clearly dominant. In particular, compared with low-lag BVAR and DSGE-VAR, its forecast errors are usually close rather than systematically lower. Therefore, the baseline horse race supports a limited conclusion: the DSGE prior can make an overparameterized neural network stable and competitive in small-sample macroeconomic forecasting, but I do not find that DSGE-NN-MH clearly outperforms low-lag BVAR or DSGE-VAR.

5.3 Forecast Paths: What Does the Neural Network Learn?

To understand why DSGE-NN-MH does not clearly outperform BVAR and DSGE-VAR, I next compare the out-of-sample forecast paths. This figure is not mainly intended to compare error magnitudes again. Instead, it examines whether the models generate materially different forecasting patterns.

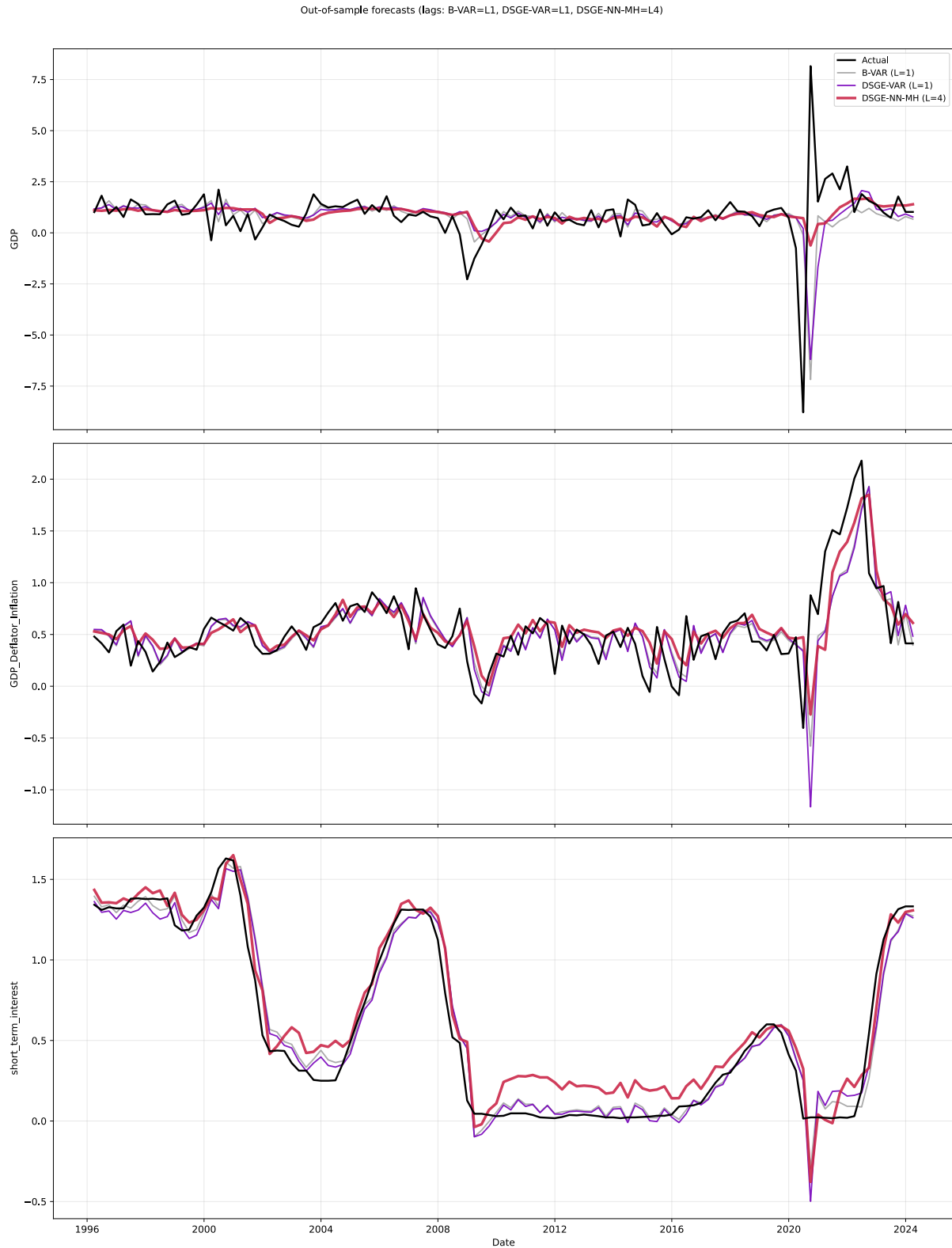


Figure 3: **Out-of-sample forecast paths.** The three panels correspond to GDP growth, GDP deflator inflation, and the short-term interest rate. The black line reports realized values, while the other lines report one-step-ahead forecasts from different models during the test period.

Figure 3 shows that the forecast path of DSGE-NN-MH is very close to those of BVAR and DSGE-VAR. This suggests that the neural network does not learn a sharply different nonlinear forecasting rule. Instead, its forecasting pattern is highly similar to the signal captured by linear

Bayesian predictors. This is central to the interpretation of the paper: although DSGE-NN-MH uses an overparameterized neural network, it appears to learn a low-dimensional, smooth, approximately linear Bayesian-shrinkage-like signal rather than an additional nonlinear predictive signal that traditional linear models fail to capture.

This finding also explains the horse-race results. DSGE-NN-MH can use pretraining and the bottleneck to avoid the instability of high-lag specifications. But if the stable predictable component in the data is already largely captured by low-lag BVAR or DSGE-VAR, the more flexible neural network need not deliver a large reduction in forecast errors.

5.4 Robustness: Seeds, Information Set, and Architecture

I next examine whether the main conclusion is driven by neural-network randomness or by a specific choice of information set or architecture.

First, Figure 4 fixes the DSGE-pretrained weights and varies the fine-tuning random seed. This exercise checks whether the main result is driven by a lucky seed or stochastic optimization noise.

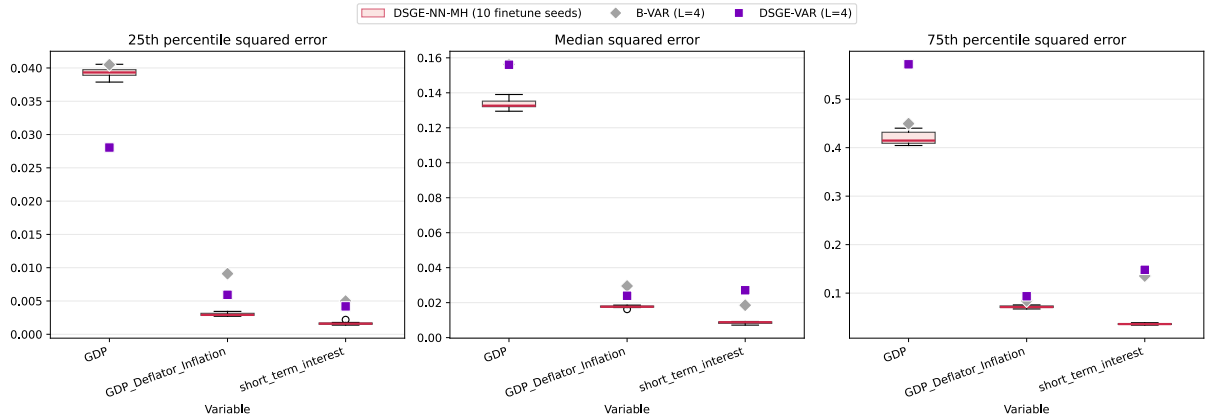


Figure 4: **Fine-tuning seed sensitivity.** The figure compares squared forecast error distributions at $L = 4$ across different fine-tuning random seeds. The DSGE-pretrained weights are fixed, and only the fine-tuning seed is varied.

The forecast error distributions are concentrated across seeds. This suggests that the main result is not primarily driven by random initialization or optimization noise. In other words, the finding that DSGE-NN-MH does not clearly outperform low-lag Bayesian benchmarks is unlikely to be an accident of one failed training run.

Second, I consider an extended information set. In this specification, the output variables remain GDP growth, GDP deflator inflation, and the short-term interest rate, but the input is expanded to include lags of eight macroeconomic variables. The purpose is not to replace the baseline model, but to ask whether giving the neural network more macroeconomic information and a richer DSGE-related source domain generates stronger predictive signals.

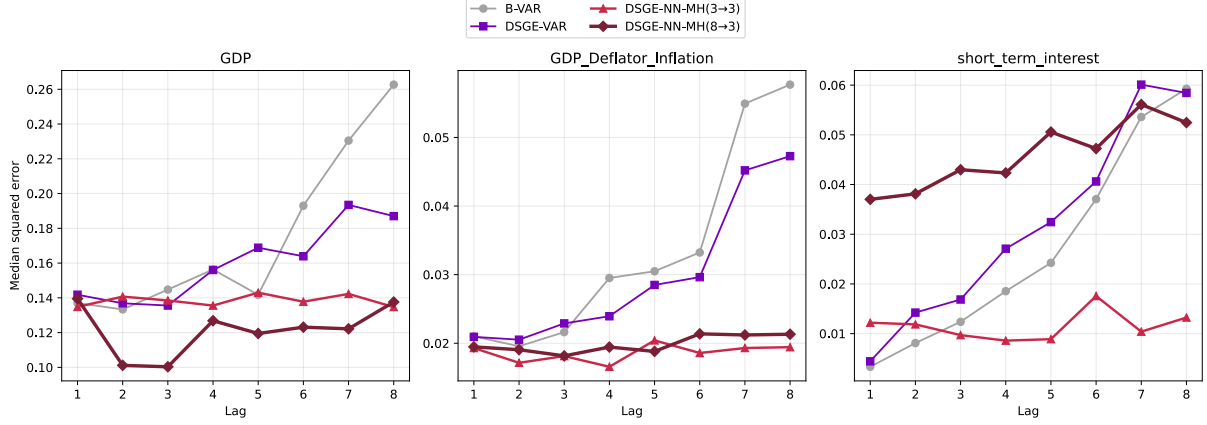


Figure 5: **Extended information set: median squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The output variables remain the three baseline targets, while the input variables are expanded to eight macroeconomic variables. Each line reports the median squared forecast error across test-period forecast origins.

Figure 5 shows that the extended information set does not systematically improve forecast performance. This should not be interpreted as evidence that high-dimensional information is generally useless, since the current extended information set is still limited and the sample remains short. Rather, the result says that, in the present implementation, simply adding more low-frequency macroeconomic variables does not generate clear forecast dominance for DSGE-NN-MH.

Finally, I compare the bottleneck multi-head architecture with a standard fully connected neural network. Both models are DSGE-pretrained, so this comparison asks whether the result depends on the specific architecture.

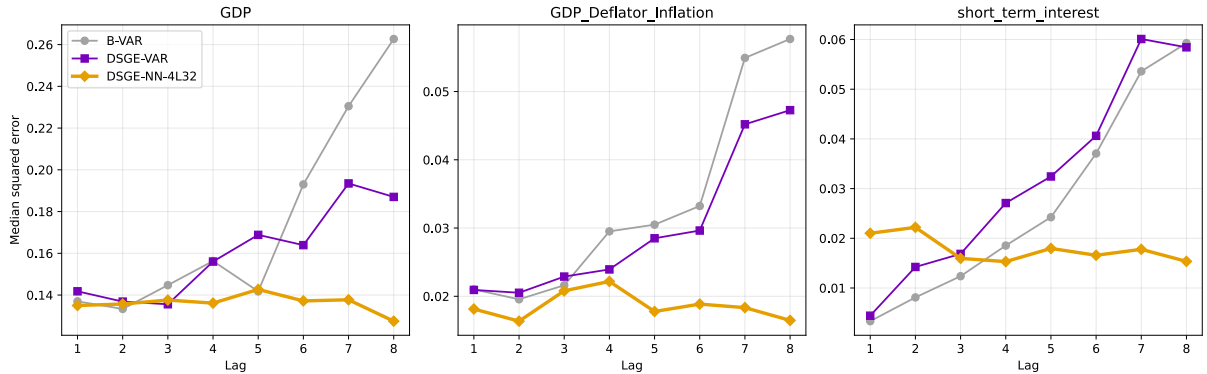


Figure 6: **Architecture comparison: median squared forecast error.** The figure compares a DSGE-pretrained bottleneck multi-head network with a DSGE-pretrained fully connected neural network. The horizontal axis reports lag length, and the vertical axis reports median squared forecast error.

Figure 6 shows that the two DSGE-pretrained neural architectures perform very similarly. This suggests that, in the current implementation, the main stabilizing force is more likely the DSGE-induced initialization than the exact neural-network architecture. Taken together, the seed-sensitivity, extended-input, and architecture-comparison results indicate that the main conclusion is not primarily driven by one random seed, one information set, or the bottleneck multi-head architecture alone.

5.5 What Does the DSGE Prior Do?

The final empirical question is what the DSGE prior actually contributes. The previous results show that DSGE-NN-MH can become stable and competitive with Bayesian benchmarks. The remaining question is whether this prior provides additional structural strength that allows the neural network to learn a new predictor unavailable to linear models.

To address this question, I compare the DSGE prior with a historical-mean prior, a Minnesota-style random-walk prior, and a no-prior raw NN. All neural-network variants use the same architecture and rolling fine-tuning procedure. They differ only in the source-domain process used for pretraining. Figure 7 compares median squared errors across fine-tuning epochs.

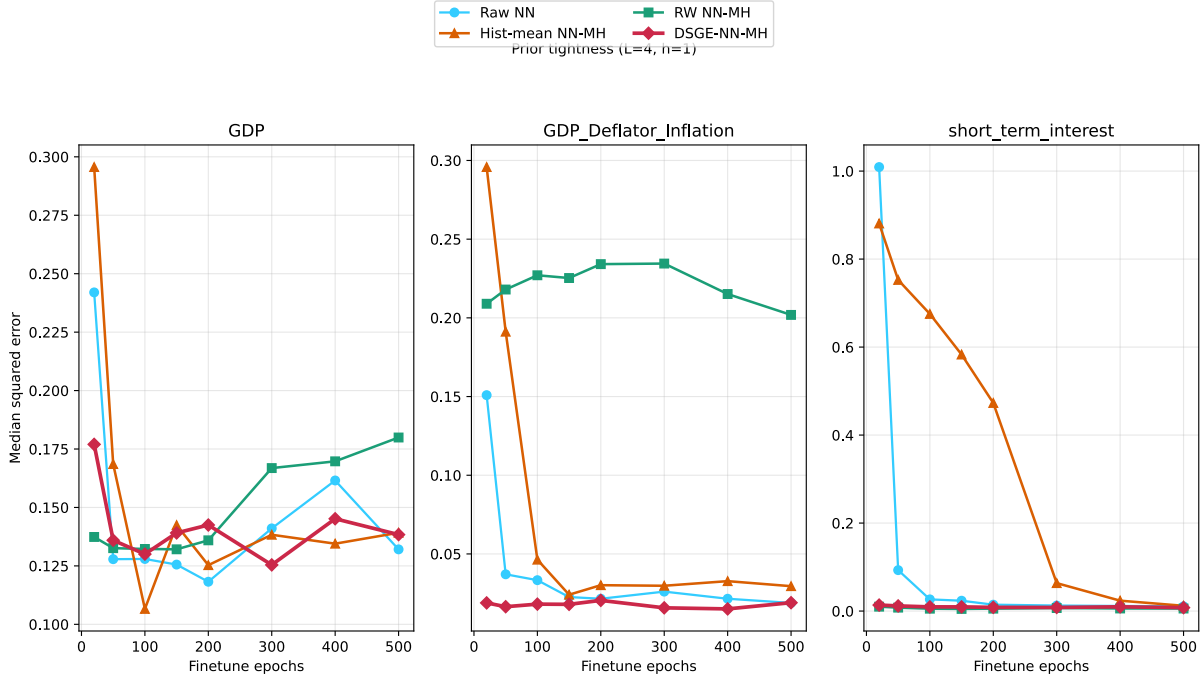


Figure 7: **Prior comparison: median squared forecast error versus fine-tuning epochs.** The horizontal axis reports the number of fine-tuning epochs. Each panel corresponds to one target variable. Different lines correspond to different pretraining priors or the raw NN.

The results show that the DSGE prior is useful. Relative to the raw NN, DSGE pretraining moves the model into a stable forecasting region more quickly. Relative to the historical-mean and random-walk priors, the DSGE prior is better aligned with macroeconomic dynamics. The historical-mean prior tends to pull persistent variables toward their long-run means, which can be particularly misleading for the interest rate. The random-walk prior captures persistence and the no-change forecast idea, but it lacks cross-variable dynamics and shock propagation. The raw NN depends heavily on small-sample fine-tuning and is therefore more unstable.

Figure 8 shows forecast paths under different priors. This helps reveal the forecasting patterns behind the numerical errors.

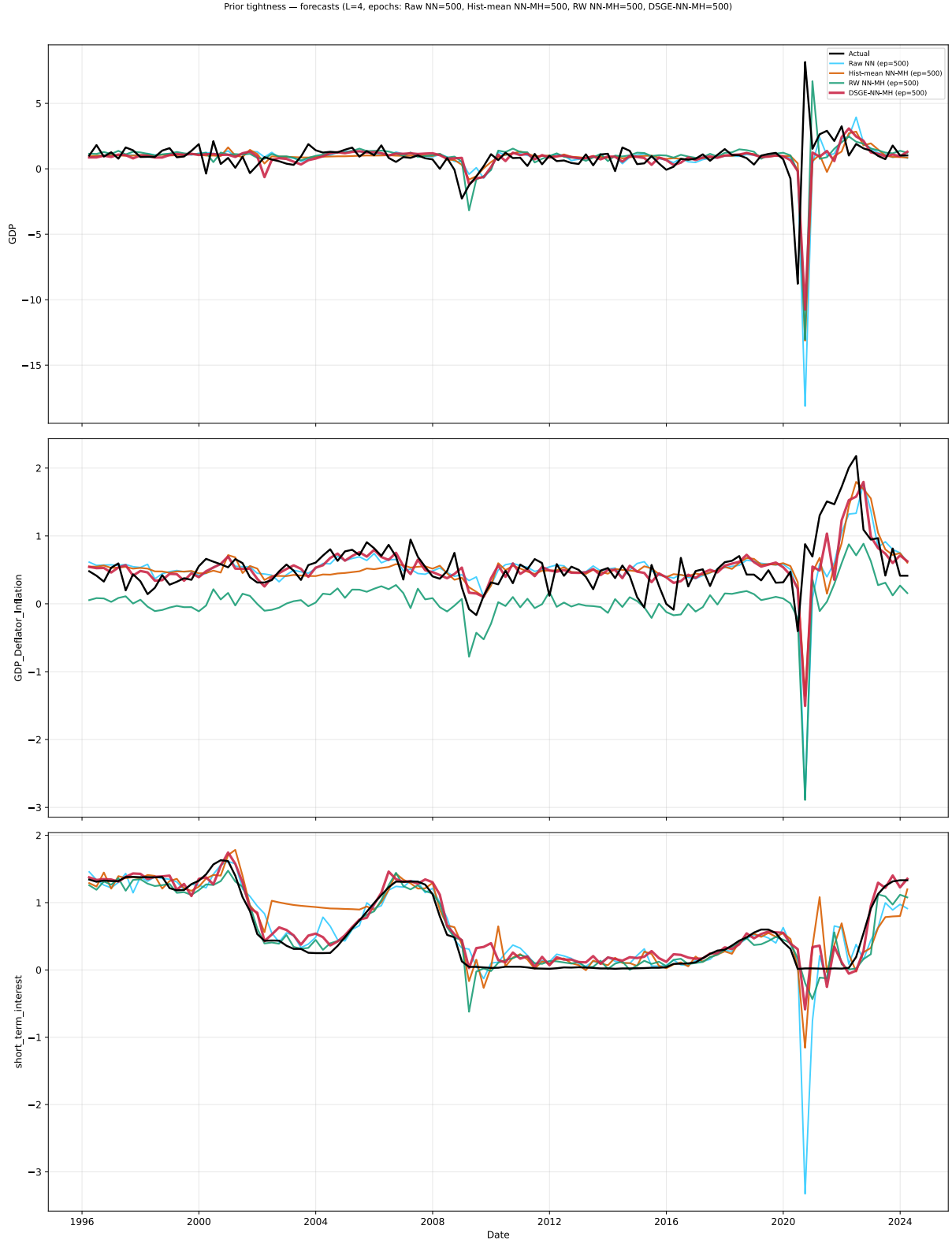


Figure 8: **Prior comparison: forecast paths at selected fine-tuning epochs.** The figure reports out-of-sample forecast paths for different pretraining priors and the raw NN at selected fine-tuning epochs. The three panels correspond to the three target variables.

These results indicate that the value of the DSGE prior is not merely that any pretraining stabilizes the network. The quality of the prior center matters. DSGE pretraining places the network closer to a reasonable macroeconomic forecasting region, allowing fine-tuning to reach

smooth forecasts more quickly and more reliably.

At the same time, the role of the DSGE prior should be defined cautiously. I do not find strong evidence of a conditional bias–variance tradeoff in which an intermediate prior tightness clearly preserves both structural strength and data-driven flexibility and thereby systematically outperforms BVAR or DSGE-VAR. Instead, the evidence supports a weaker but more robust interpretation: the DSGE prior mainly provides a good starting point. It helps the overparameterized neural network learn low-dimensional, approximately linear forecasting signals that are very similar to those captured by BVAR and DSGE-VAR. It helps the model reach a forecast region that existing benchmarks can also reach, rather than guiding the model toward a clearly superior new nonlinear predictor.

5.6 Summary of Empirical Evidence

Overall, the empirical evidence supports a cautious conclusion. DSGE-informed transfer learning can make an overparameterized neural network stable and competitive in small-sample macroeconomic forecasting, especially at higher lag lengths. However, it does not clearly outperform low-lag BVAR or DSGE-VAR. The forecast paths, seed robustness, extended information set, architecture comparison, and prior comparison all point to the same interpretation: the main role of DSGE-NN-MH is closer to regularization and representation stabilization than to the discovery of additional nonlinear predictive signals unavailable to linear Bayesian predictors.

6 Conclusion

This paper studies whether a DSGE prior can help an overparameterized neural network outperform traditional Bayesian benchmarks in small-sample macroeconomic forecasting. The results show that the DSGE prior does improve the stability of the neural network. In particular, as lag length increases, the forecast errors of DSGE-NN are less likely to deteriorate than those of BVAR and DSGE-VAR. However, this stability does not translate into systematic forecast dominance. Relative to low-lag BVAR and DSGE-VAR, DSGE-NN is usually competitive, but not clearly superior.

The forecast paths further show that DSGE-NN produces predictions that are highly similar to those of BVAR and DSGE-VAR. This suggests that the neural network mainly learns a low-dimensional, smooth, and approximately linear Bayesian-shrinkage-like signal, rather than an additional nonlinear predictor that traditional linear models fail to capture. The seed robustness, extended information set, and architecture comparison exercises all support this interpretation.

Therefore, the role of the DSGE prior should be interpreted cautiously. It is useful relative to a raw neural network, a historical-mean prior, and a random-walk prior, but its main contribution is regularization and prior-induced initialization rather than clear additional structural strength. Future research that aims to exploit the advantages of neural networks in macroeconomic forecasting may need to use higher-dimensional and higher-frequency information sets, and to develop Bayesian transfer learning or probabilistic neural forecasting methods that characterize both point forecasts and forecast uncertainty.

References

- Belkin, M., Hsu, D., Ma, S., and Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854.
- Bok, B., Caratelli, D., Giannone, D., Sbordone, A. M., and Tambalotti, A. (2018). Macroeconomic nowcasting and forecasting with big data. *Annual Review of Economics*, 10(1):615–643.
- Del Negro, M. and Schorfheide, F. (2004). Priors from general equilibrium models for vars. *International Economic Review*, 45(2):643–673.
- Doan, T., Litterman, R., and Sims, C. (1984). Forecasting and conditional projection using realistic prior distributions. *Econometric reviews*, 3(1):1–100.
- Gal, Y. and Ghahramani, Z. (2016). Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR.
- Giannone, D., Lenza, M., and Primiceri, G. E. (2015). Prior selection for vector autoregressions. *Review of Economics and Statistics*, 97(2):436–451.
- Herbst, E. P. and Schorfheide, F. (2015). *Bayesian Estimation of DSGE Models*. Princeton University Press, Princeton, NJ.
- Litterman, R. B. (1986). Forecasting with bayesian vector autoregressions—five years of experience. *Journal of Business & Economic Statistics*, 4(1):25–38.
- Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., and Sutskever, I. (2021). Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003.
- Sims, C. A. (1980). Macroeconomics and reality. *Econometrica: journal of the Econometric Society*, pages 1–48.
- Sims, C. A. (2002). Solving linear rational expectations models. *Computational economics*, 20(1-2):1.
- Stock, J. H. and Watson, M. W. (2002). Macroeconomic forecasting using diffusion indexes. *Journal of business & economic statistics*, 20(2):147–162.

A Additional Details on DSGE Estimation and Simulation

This appendix reports the technical details behind Section 2. These details are not essential for the main forecasting argument, but they document the precise Bayesian estimation and simulation procedure used to construct the DSGE prior.

A.1 MCMC Run Settings

The posterior distribution of the NK DSGE parameters is estimated using the three observable series ordered as

$$Y_t = (YGR_t, INFL_t, INT_t)'$$

The estimation sample is the portion of `estimation_data.csv` strictly before 1990Q1. Table 2 reports the main MCMC settings.

Table 2: MCMC settings

Setting	Value
Number of chains	4
Draws per chain	100,000
Burn-in per chain	5,000
Thinning interval	4
Retained draws per chain	23,750
Random seed	2026
Observable ordering	YGR, INFL, INT

The acceptance rates are reported in Table 3. They are close to 23 percent across chains, which is within a reasonable range for a random-walk Metropolis–Hastings sampler in this DSGE setting.

Table 3: Metropolis–Hastings acceptance rates

Chain	Acceptance rate
1	0.2305
2	0.2289
3	0.2321
4	0.2309

A.2 Parameter Transformations

The Metropolis–Hastings sampler is implemented in an unconstrained parameter space. Let $x \in R^{13}$ denote the transformed vector. The mapping from x to structural parameters $\theta = T(x)$ is defined as follows:

$$\theta_j = \exp(x_j), \quad \text{for positive parameters,} \quad (28)$$

$$\rho_j = \frac{1}{1 + \exp(-x_j)}, \quad \text{for persistence parameters,} \quad (29)$$

$$\gamma_Q = x_j, \quad \text{for steady-state quarterly growth.} \quad (30)$$

The positive parameters are

$$\tau, \kappa, \psi_1, \psi_2, r_A, \pi_A, \sigma_R, \sigma_g, \sigma_z,$$

and the persistence parameters are

$$\rho_R, \rho_g, \rho_z.$$

Since the prior is specified over the structural parameter vector θ rather than the transformed vector x , the transformed-space posterior includes the log-Jacobian term

$$\log \left| \frac{\partial \theta}{\partial x} \right|.$$

For log-transformed parameters, the contribution is x_j . For logit-transformed persistence parameters, the contribution is

$$\log \rho_j + \log(1 - \rho_j).$$

A.3 Prior Specification

The prior is chosen to impose economically meaningful parameter restrictions while remaining sufficiently diffuse for the data to update the structural parameters. Table 4 summarizes the prior families.

Table 4: Prior distributions for the NK DSGE parameters

Parameter group	Parameters	Prior family
Preference and Phillips-curve parameters	τ, κ	Gamma
Policy-rule coefficients	ψ_1, ψ_2	Gamma
Persistence parameters	ρ_R, ρ_g, ρ_z	Beta
Steady-state real rate and inflation	r_A, π_A	Gamma
Steady-state quarterly growth	γ_Q	Normal
Shock standard deviations	$\sigma_R, \sigma_g, \sigma_z$	Inverse-gamma-style prior

A.4 Metropolis–Hastings Algorithm

At iteration s , the current transformed parameter vector is $x^{(s)}$. The random-walk proposal is

$$x^\star = x^{(s)} + \xi_s, \quad \xi_s \sim N(0, \Sigma_q).$$

Because the proposal density is symmetric, the Metropolis–Hastings acceptance probability is

$$\alpha(x^{(s)}, x^\star) = \min \left\{ 1, \frac{p(x^\star | Y)}{p(x^{(s)} | Y)} \right\}.$$

Equivalently, in log terms, the proposal is accepted if

$$\log u < \log p(x^\star | Y) - \log p(x^{(s)} | Y), \quad u \sim U(0, 1).$$

The proposal covariance matrix is diagonal and deliberately conservative, since many proposed parameter values can violate the Blanchard–Kahn conditions or generate numerically unstable Kalman likelihoods.

A.5 Kalman Filter Initialization

For each admissible parameter vector, the DSGE solution implies the linear Gaussian state-space system

$$a_t = T a_{t-1} + R \varepsilon_t, \quad Y_t = d + Z a_t + u_t.$$

The Kalman filter is initialized at the unconditional covariance matrix of the state vector. When the transition matrix is stable, this covariance matrix solves the discrete Lyapunov equation

$$P_0 = TP_0T' + RQR'.$$

Parameter values that fail to deliver a stable solution or a finite likelihood are assigned log posterior equal to $-\infty$ and are automatically rejected by the sampler.

A.6 Posterior Diagnostics

Table 5 reports posterior diagnostics from the retained draws. The $\hat{R}$ statistics are approximately one for all parameters, suggesting no obvious cross-chain convergence problem. The bulk effective sample size is lowest for ψ_2 , which indicates that inference on the output-gap response in the Taylor rule is less precise than for the other parameters.

Table 5: Posterior diagnostics

Parameter	Mean	SD	ESS bulk	$\hat{R}$
τ	2.7700	0.5300	1368	1.00
κ	0.5080	0.1380	1106	1.00
ψ_1	1.1310	0.0850	4436	1.00
ψ_2	0.0900	0.0730	942	1.00
ρ_R	0.6310	0.0510	1807	1.00
ρ_g	0.9800	0.0100	1354	1.00
ρ_z	0.8620	0.0210	1793	1.00
r_A	0.3030	0.1450	2579	1.00
π_A	2.6800	0.4900	1113	1.00
γ_Q	0.2560	0.0780	1965	1.00
σ_R	0.002666	0.000226	3446	1.00
σ_g	0.013710	0.001070	5453	1.00
σ_z	0.002124	0.000258	3028	1.00

A.7 Posterior Output

The posterior simulation produces raw chains in transformed space, post-burn-in draws in transformed space, and post-burn-in draws in the original structural parameterization. The retained posterior draws have shape

$$(4, 23750, 13),$$

corresponding to four chains, 23,750 retained draws per chain, and 13 structural parameters. The posterior mean vector is saved as `theta.csv` and is used as the fixed parameter point $\bar{\theta}$ in the downstream DSGE simulation, DSGE-VAR benchmark, and DSGE-informed neural network pretraining.

A.8 Simulation Algorithm

Given the posterior mean $\bar{\theta}$, the artificial DSGE sample is generated as follows. First, construct a path-specific parameter vector θ_i by applying a small perturbation around $\bar{\theta}$ in the transformed parameter space. Second, solve the linear rational expectations model for θ_i . Third, simulate the state transition equation for 200 quarters using independent Gaussian structural shocks. Fourth, discard the first 100 quarters as burn-in. Fifth, map the simulated states into observables using the measurement equations. Sixth, convert annualized inflation and interest rates into quarterly

units by dividing them by four. Finally, rescale each simulated variable using the pre-1980 mean and standard deviation of the corresponding real data series.

The resulting file, `simulation_3var.csv`, contains 1,000 simulated paths. For each path, lags one through eight are constructed internally so that the simulated sample can be used directly to pretrain forecasting models with different lag lengths.

B Additional Details on the DSGE-NN-MH Forecasting Model

This appendix reports implementation details for the DSGE-NN-MH model described in Section 3. The main text focuses on the economic role of DSGE pretraining and the interpretation of the forecasting results. The details below are included for reproducibility.

B.1 Baseline Architecture

The baseline DSGE-NN-MH architecture consists of a shared encoder, a shared decoder, and three variable-specific output heads. The encoder maps the lagged input vector into a four-dimensional bottleneck representation. The decoder then expands this representation before the variable-specific heads produce the final forecasts.

Table 6: Baseline DSGE-NN-MH architecture

Component	Specification
Input	Lagged macro variables up to $L_{\max} = 8$
Encoder hidden layers	$32 \rightarrow 32$
Bottleneck dimension	4
Shared decoder width	32
Number of output heads	3
Each output head	$16 \rightarrow 1$
Activation function	ReLU
Dropout rate	0.1

The three output heads correspond to GDP growth, GDP deflator inflation, and the short-term interest rate. In the baseline three-variable experiment, the input variables are the same three series. In the extended-input experiment, the input contains eight variables, while the output remains the same three target variables.

B.2 Training Hyperparameters

Table 7 reports the baseline training settings. The pretraining stage uses the DSGE-simulated sample, while the fine-tuning stage uses rolling windows of real data.

The validation period is used to choose the main neural-network settings. The final test results use the same hyperparameters across lag lengths and forecast origins. No test-period tuning is performed.

B.3 Rolling-Origin Forecasting Procedure

For each forecast origin T , the rolling forecasting procedure is:

1. Construct the real-data fine-tuning sample using observations from $T - W$ through $T - 1$, where $W = 80$.
2. Initialize the neural network at the DSGE-pretrained weights.

Table 7: Baseline DSGE-NN-MH training settings

Setting	Value
Forecast horizon	$h = 1$
Maximum lag length	$L_{\max} = 8$
Validation lag length	$L = 4$
Test lag grid	$L = 1, \dots, 8$
Pretraining learning rate	4×10^{-4}
Pretraining epochs	100
Pretraining batch size	256
Fine-tuning learning rate	1.5×10^{-4}
Fine-tuning epochs	200
Fine-tuning batch size	16
Rolling estimation window	80 quarters
Freeze encoder	No
Fine-tuning early stopping	No
Baseline random seed	42

3. Fine-tune the network on the rolling real-data window.
4. Use the fine-tuned network to forecast Y_{T+1} from $X_{T,L}$.
5. Move the forecast origin forward by one quarter and repeat the procedure.

This procedure is computationally more expensive than estimating a single neural network once, but it aligns the neural-network evaluation with the rolling pseudo-out-of-sample design used for the BVAR and DSGE-VAR benchmarks.

B.4 Input Construction and Lag Handling

The maximum lag length is set to $L_{\max} = 8$. The neural-network architecture is kept fixed across all lag lengths. For a given lag length $L < L_{\max}$, the input vector is padded with zeros in the positions corresponding to unused lags. This convention keeps the number of trainable parameters constant across $L = 1, \dots, 8$. Therefore, the lag-length comparison mainly reflects the informational content and variance cost of adding lagged predictors, rather than changes in architecture size.

For the baseline experiment, the input vector contains lags of the three target variables:

$$Y_t = (GDP_t, \pi_t, i_t)'$$

For the extended-input experiment, the input vector contains lags of eight macroeconomic variables:

$$Z_t = (GDP_t, \pi_t^{GDP}, \pi_t^{CorePCE}, i_t^{short}, \Delta Inv_t, Spread_t, i_t^{long}, \Delta Cons_t)'$$

The output remains

$$Y_t = (GDP_t, \pi_t, i_t)'$$

Thus the extended-input experiment changes the information set but not the forecasting target.

B.5 Seed Sensitivity

The baseline DSGE-NN-MH model uses random seed 42. To check whether the main findings are driven by neural-network optimization randomness, I also run a seed-sensitivity exercise. In

that exercise, the DSGE-pretrained weights are fixed, and only the fine-tuning random seed is varied. The seed grid is

$$\{42, 123, 456, 789, 1337, 2026, 3407, 5678, 8765, 9999\}.$$

The exercise is conducted at lag length $L = 4$. The resulting forecast error distributions are compared with the corresponding BVAR and DSGE-VAR benchmarks.

B.6 Interpretation of Pretraining as an Implicit Prior

The DSGE-NN-MH model does not impose a closed-form Bayesian prior directly on the neural-network weights. Instead, the prior enters through the optimization path. Pretraining on DSGE simulations maps the randomly initialized network to a parameter vector

$$(\hat{\phi}^{pre}, \hat{\omega}^{pre}),$$

where ϕ denotes the shared representation parameters and ω denotes the output-head parameters. These parameters induce a pretrained forecasting function

$$f_{\hat{\phi}^{pre}, \hat{\omega}^{pre}}.$$

Fine-tuning then updates this function using real data. The effective tightness of the prior is governed by how far optimization is allowed to move away from the pretrained weights. In practice, this depends on the learning rate, number of fine-tuning epochs, batch size, dropout, and whether any layers are frozen.

This interpretation is weaker than a fully Bayesian neural-network formulation. The model does not define an explicit prior density over neural-network weights, and it does not integrate over posterior uncertainty in those weights. Nevertheless, the interpretation is useful for this paper’s purpose: DSGE simulations discipline the initial forecasting function, while real data determine how much that function is revised in the target forecasting task.

C Additional Details on Benchmark Models

This appendix records implementation details for the benchmark models in Section 4. It avoids repeating the main economic interpretation and focuses only on choices needed for replication.

C.1 Minnesota Prior Details

For the BVAR, let $B_{\ell,ij}$ denote the coefficient on variable j at lag ℓ in the equation for variable i . The prior mean is

$$E[B_{\ell,ij}] = \begin{cases} 1, & i = j, \ell = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Thus each variable is shrunk toward own first-lag persistence, while cross-variable and longer-lag coefficients are centered at zero.

The prior variance is

$$\text{Var}(B_{\ell,ij}) = \lambda_B^2 \frac{1}{\ell^2} \frac{\hat{\sigma}_i^2}{\hat{\sigma}_j^2} \times \begin{cases} 1, & i = j, \\ \chi, & i \neq j, \end{cases}$$

where $\hat{\sigma}_i^2$ is a scale estimate from a univariate autoregression for variable i , and $\chi \in (0, 1]$ controls the relative tightness on cross-variable coefficients. The intercept receives a diffuse prior.

The validation grid for the overall tightness parameter is

$$\lambda_B \in \{0.05, 0.10, 0.20, 0.30, 0.40\}.$$

The selected value is fixed before the final test-period evaluation.

C.2 DSGE-VAR Implementation Details

For each lag length L , the simulated DSGE sample is converted into the same regressor-target format as the real forecasting sample:

$$\mathcal{D}_L^{sim} = \{(X_{t,L}^{sim}, Y_{t+1}^{sim})\}.$$

The simulated moment matrices are computed once for each L and then reused across rolling forecast origins. The real-sample moment matrices are recomputed at each forecast origin using the 80-quarter rolling window.

The validation grid for the DSGE prior tightness parameter is

$$\lambda_D \in \{0.2, 0.4, 0.6, 0.8, 1.0\}.$$

The selected value is fixed during the final test-period evaluation.

The use of simulated sample moments is a practical implementation choice. It aligns DSGE-VAR with DSGE-NN-MH by giving both models the same source-domain information. It also avoids introducing a separate analytical population-moment computation that is not used elsewhere in the paper.

C.3 Raw Neural Network Benchmark

The raw NN uses the same architecture, rolling window, and loss function as DSGE-NN-MH. The only difference is initialization:

DSGE-NN-MH: initialized at DSGE-pretrained weights,

Raw NN: initialized randomly at each forecast origin.

The baseline raw NN uses

$$W = 80, \quad \text{training epochs} = 200, \quad \text{batch size} = 16.$$

This keeps the comparison focused on the value of DSGE pretraining rather than on model size or training length.

C.4 Additional Neural Prior Variants

For the prior-comparison exercise, I use the same neural architecture and fine-tuning procedure but change the source-domain pretraining data.

The historical-mean prior generates source-domain targets centered around the pre-sample mean of each variable. This prior is useful as a simple benchmark for mean-reverting behavior, but it can be poorly aligned for highly persistent macroeconomic variables.

The random-walk prior generates source-domain dynamics corresponding to a no-change forecast. This prior captures persistence but does not encode cross-variable macroeconomic dynamics or DSGE-style shock propagation.

These variants are used only for mechanism checks. They help distinguish the effect of DSGE-specific structural information from the generic stabilizing effect of pretraining.

D Additional Empirical Details and Supplementary Figures

This appendix records empirical details and supplementary figures not reported in Section 5. The main text keeps only the figures needed for the central argument. The figures below check whether the same conclusions hold across other error quantiles, lag choices, and forecast paths.

D.1 Empirical Design Details

The baseline target vector is

$$Y_t = (GDP_t, \pi_t, i_t)'$$

The baseline input consists of lags of these same three variables. The extended information set keeps the output variables unchanged but expands the input vector to

$$Z_t = (GDP_t, \pi_t^{GDP}, \pi_t^{CorePCE}, i_t^{short}, \Delta Inv_t, Spread_t, i_t^{long}, \Delta Cons_t)'$$

Thus, the extended-input experiment changes the information set but not the forecasting target.

The validation period is 1980Q1–1995Q4, and the test period starts in 1996Q1. At each forecast origin, all models are estimated or fine-tuned using the previous $W = 80$ quarters of data. The lag grid is

$$L = 1, \dots, 8.$$

The tightness parameters for BVAR and DSGE-VAR are selected on the validation period and then fixed during the test period. The main architecture and training settings for DSGE-NN-MH are also fixed before the test evaluation.

The evaluation metric is squared forecast error:

$$SE_{j,T}^{(m)} = \left(Y_{j,T+1} - \hat{Y}_{j,T+1|T}^{(m)} \right)^2.$$

For each variable, model, and lag length, I report quantiles across test-period forecast origins:

$$Q_{0.25}(SE_{j,T}^{(m)}), \quad Q_{0.50}(SE_{j,T}^{(m)}), \quad Q_{0.75}(SE_{j,T}^{(m)}).$$

The main text focuses on the median and upper quartile; the supplementary figures report the lower quartile and additional robustness patterns.

D.2 Baseline Horse Race: Supplementary Quantile

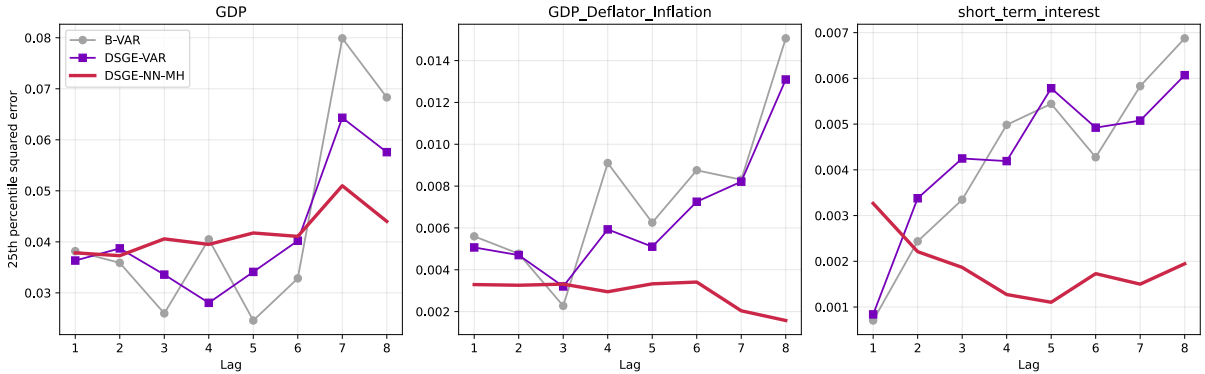


Figure 9: **Baseline horse race: 25th percentile squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Each line reports the 25th percentile squared forecast error across test-period forecast origins for one model.

D.3 Extended Information Set: Supplementary Figures

The extended information set uses FRBNY-DSGE-related data. The original FRBNY data contain thirteen macroeconomic variables. I select eight variables as inputs: GDP growth, GDP deflator inflation, core PCE inflation, the short-term interest rate, investment growth, the term

spread, the long-term interest rate, and consumption growth. The output variables remain GDP growth, GDP deflator inflation, and the short-term interest rate. The simulated source-domain data for this exercise are generated following the official FRBNY-DSGE documentation.

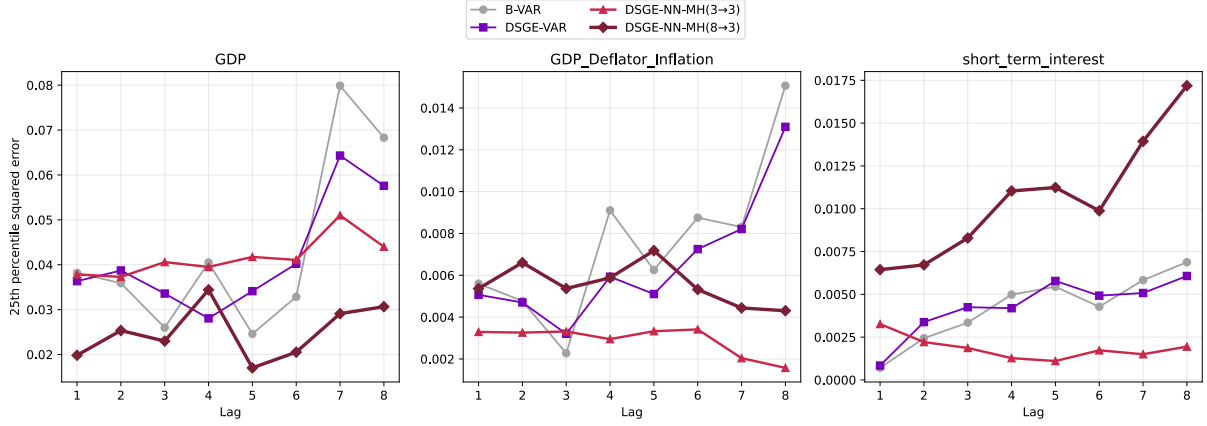


Figure 10: **Extended information set: 25th percentile squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Each line reports the 25th percentile squared forecast error under the extended-input specification.

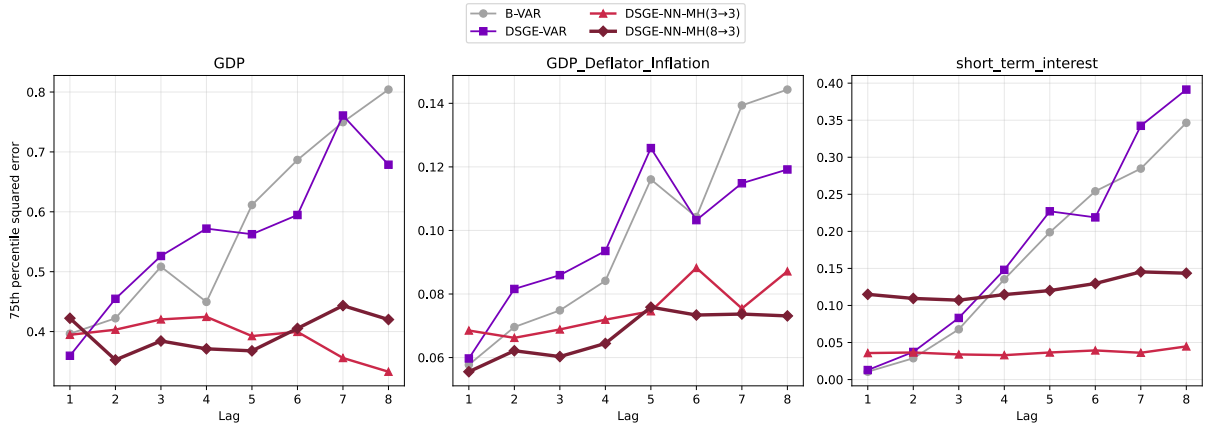


Figure 11: **Extended information set: 75th percentile squared forecast error.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Each line reports the 75th percentile squared forecast error under the extended-input specification.

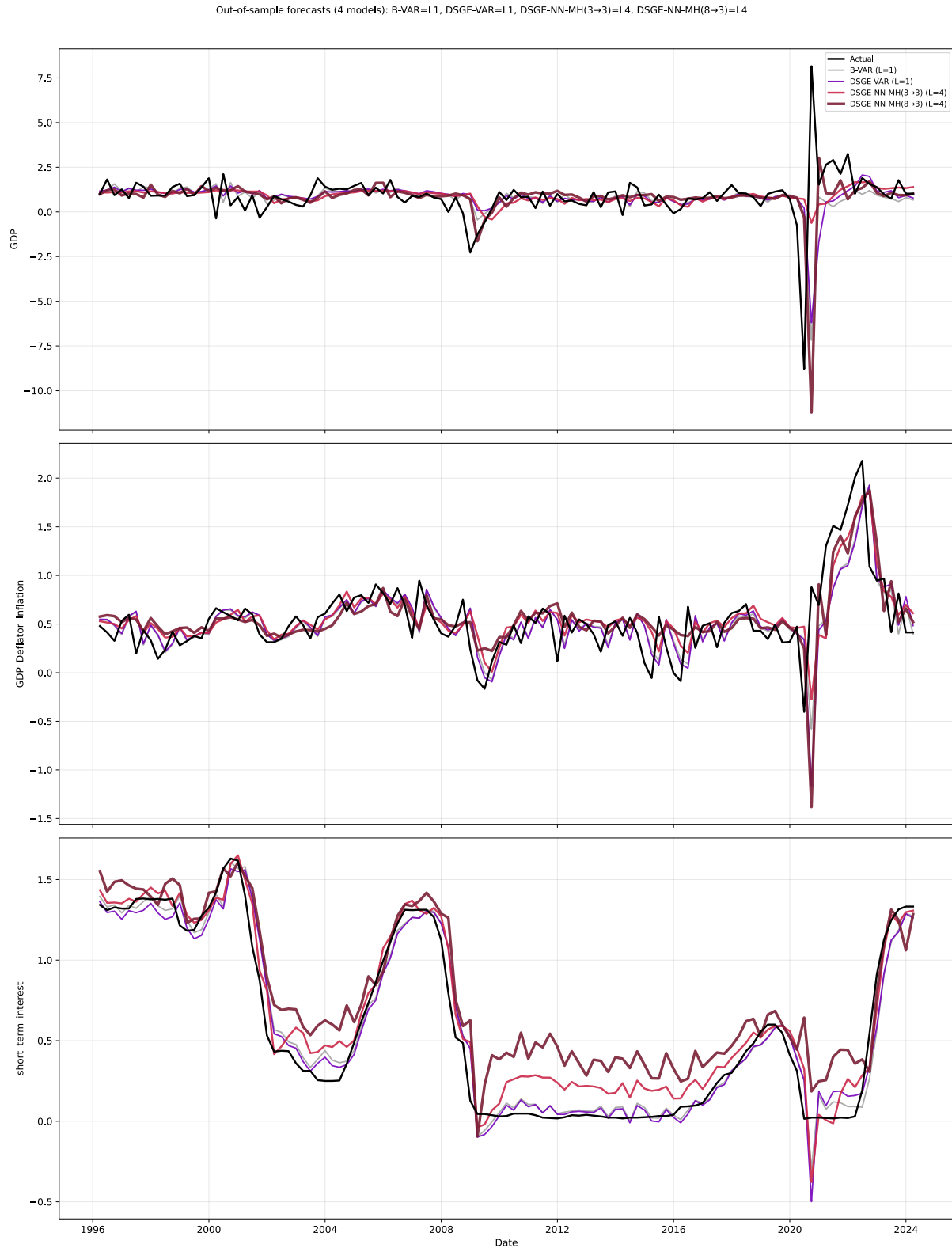


Figure 12: **Extended information set: out-of-sample forecast paths.** The three panels correspond to the three target variables. The black line reports realized values, while the other lines report one-step-ahead forecasts from different models under the extended-input specification.

D.4 Prior Comparison: Supplementary Figures

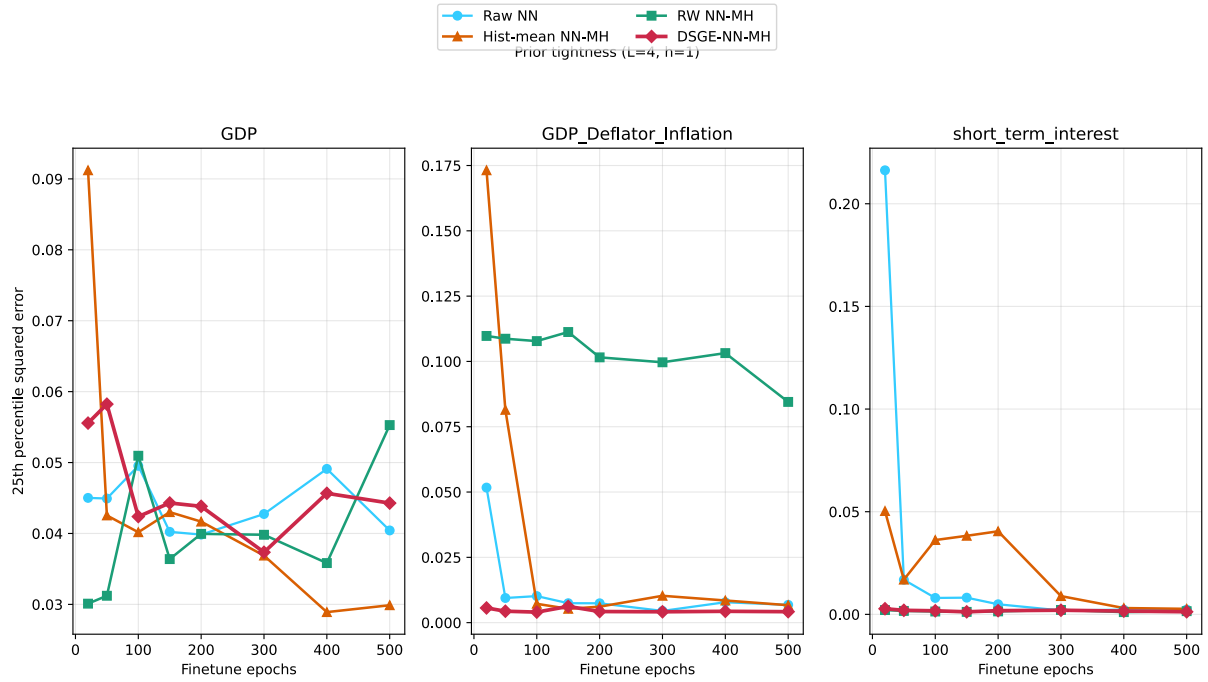


Figure 13: **Prior comparison: 25th percentile squared forecast error versus fine-tuning epochs.** The horizontal axis reports fine-tuning epochs. The three panels correspond to the three target variables. Different lines correspond to different pretraining priors or the raw NN.

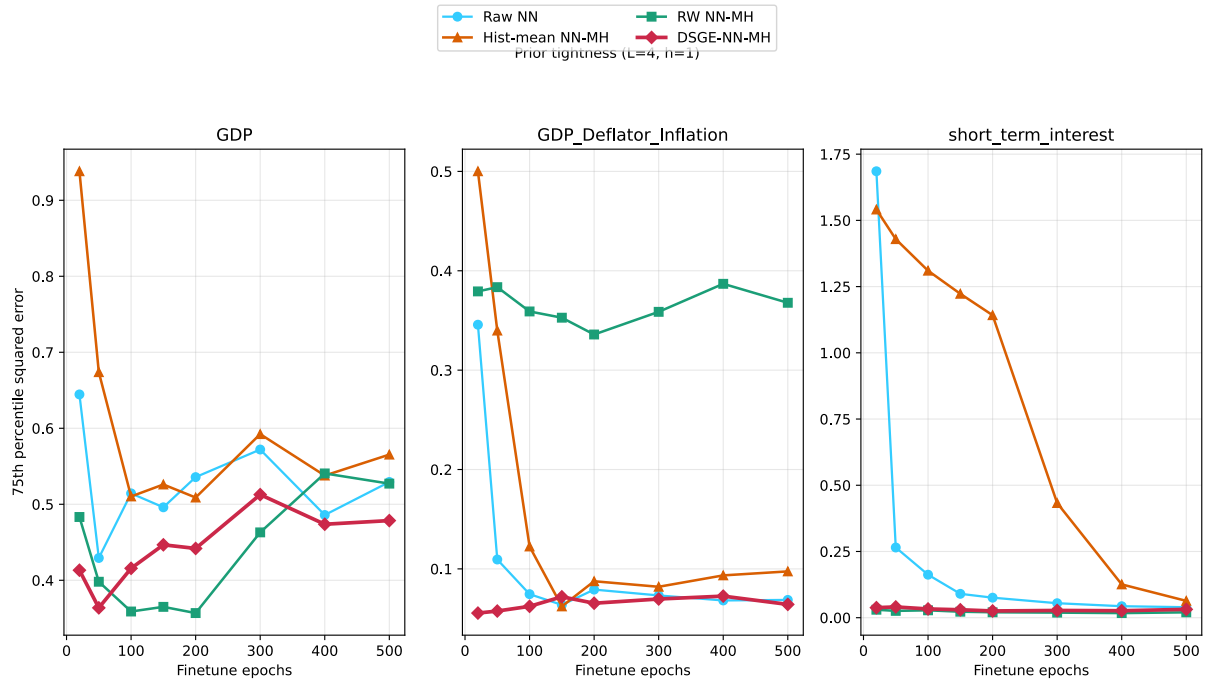


Figure 14: **Prior comparison: 75th percentile squared forecast error versus fine-tuning epochs.** The horizontal axis reports fine-tuning epochs. The three panels correspond to the three target variables. Different lines correspond to different pretraining priors or the raw NN.

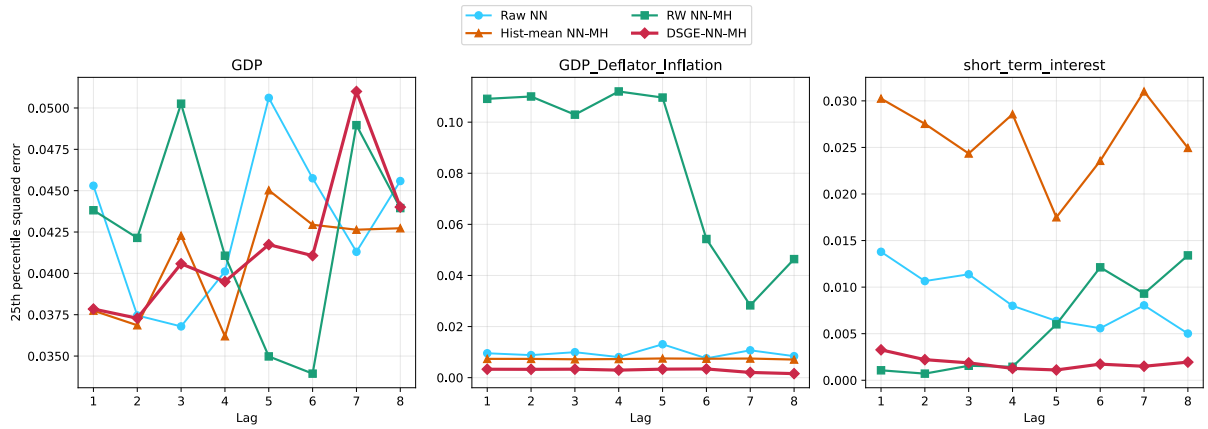


Figure 15: **Prior comparison: 25th percentile squared forecast error versus lag length.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Different lines correspond to different pretraining priors or the raw NN.

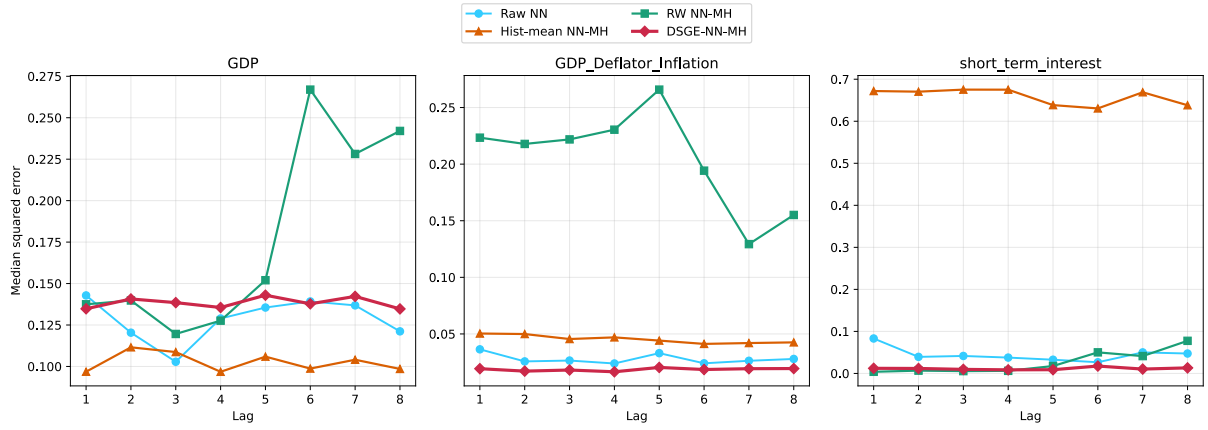


Figure 16: **Prior comparison: median squared forecast error versus lag length.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Different lines correspond to different pretraining priors or the raw NN.

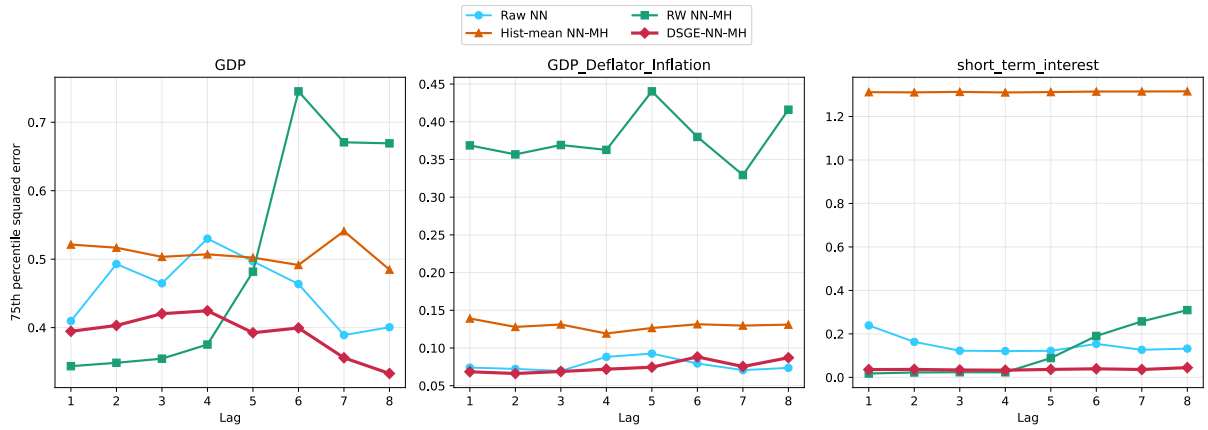


Figure 17: **Prior comparison: 75th percentile squared forecast error versus lag length.** The horizontal axis reports lag length $L = 1, \dots, 8$. The three panels correspond to the three target variables. Different lines correspond to different pretraining priors or the raw NN.

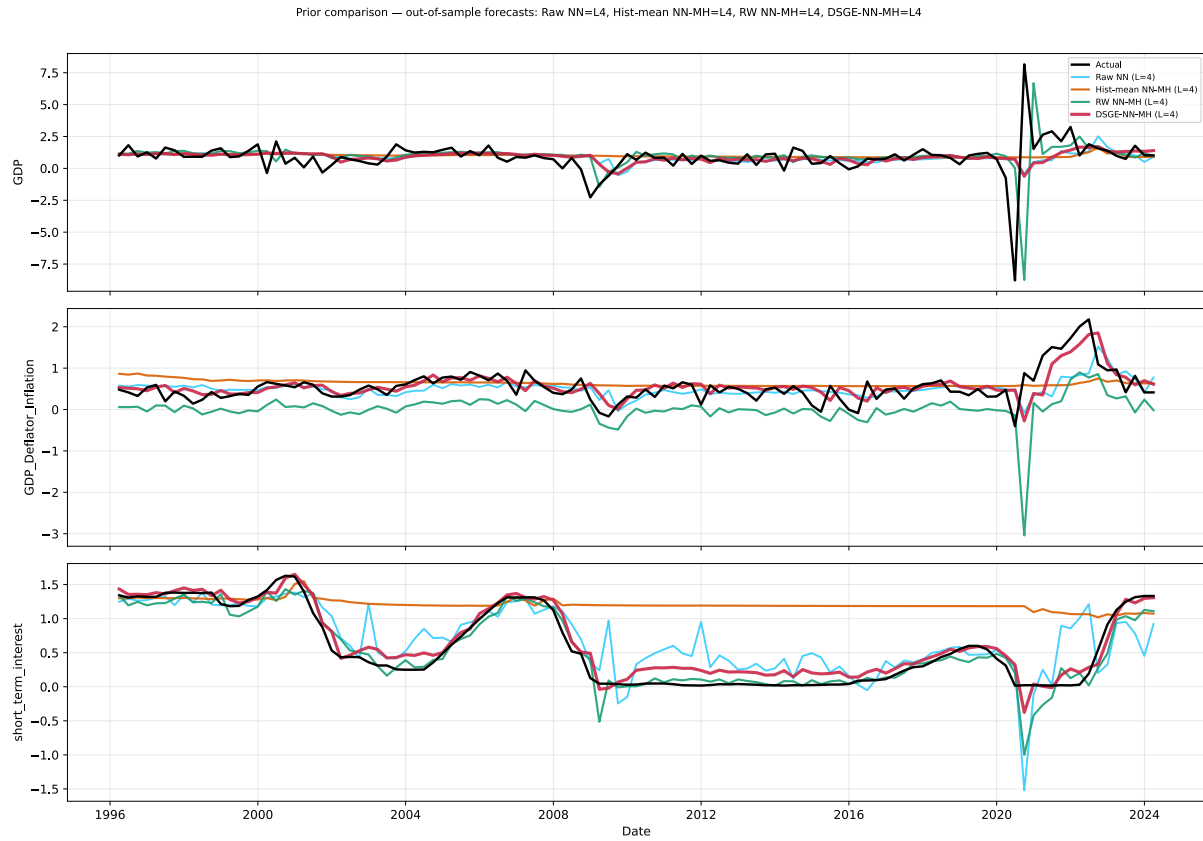


Figure 18: **Prior comparison: forecast paths at selected lag lengths.** The three panels correspond to the three target variables. The figure compares out-of-sample forecast paths from different pretraining priors or the raw NN at selected lag lengths.

D.5 Architecture Comparison: Supplementary Figure

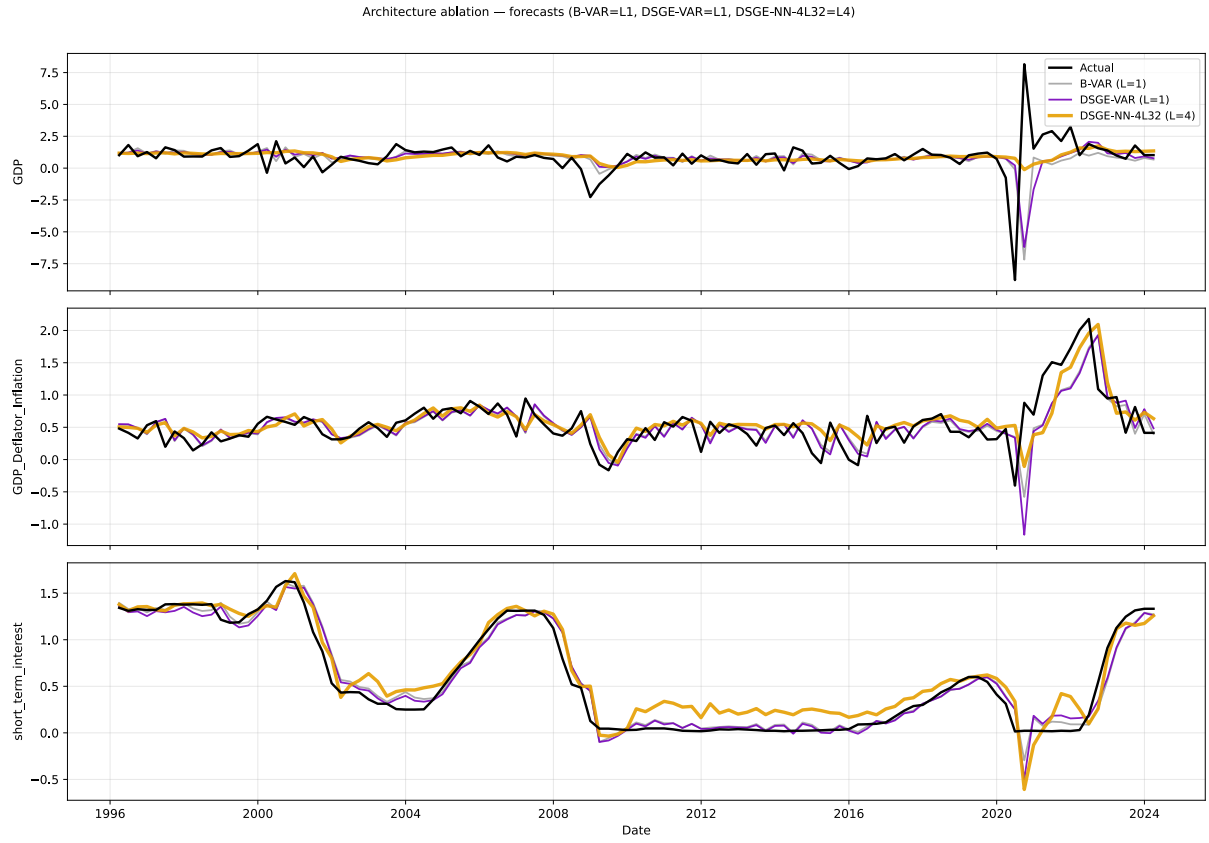


Figure 19: **Architecture comparison: out-of-sample forecast paths.** The three panels correspond to the three target variables. The figure compares one-step-ahead forecast paths from the DSGE-pretrained bottleneck multi-head network and the DSGE-pretrained fully connected neural network.